

Object Detection in the KITTI Dataset using YOLO and Faster R-CNN

Yang Gefan

College of Design and Engineering
National University of Singapore
Singapore

Email: e1101738@u.nus.edu
ORCID: 0009-0003-8334-3702

Liu Yuchi

College of Design and Engineering
National University of Singapore
Singapore

Email: e1101738@u.nus.edu
ORCID: 0009-0001-1017-1423

Abstract—Object detection is crucial in advancing autonomous driving technologies, where the complexity and unpredictability of real-world environments demand robust and accurate models. This study emphasizes the optimization of the Faster R-CNN model for object detection using the KITTI dataset, with a particular focus on detecting entities like cars, pedestrians, and cyclists. Through comprehensive training and model adjustments, Faster R-CNN has been fine-tuned to adapt to the diverse challenges present in the dataset. In a comparative evaluation, YOLOv3 and YOLOv5 were also employed, serving as benchmarks to ascertain the relative strengths and shortcomings of Faster R-CNN. Results highlighted that while Faster R-CNN achieved superior accuracy, it lagged in speed, making YOLO models more apt for real-time scenarios. This exploration aims to guide practical decisions in the selection and optimization of object detection models for real-world autonomous driving applications.

Index Terms—KITTI dataset, Faster R-CNN, YOLO, object detection, autonomous driving

I. INTRODUCTION

Autonomous driving technology is steadily becoming a pivotal feature of modern vehicles, offering the potential to improve road safety and increase traffic efficiency. A cornerstone of this technology is the object detection system, which allows a vehicle to perceive and interpret its surroundings. This capability is essential for identifying other road users, like cars, pedestrians, and cyclists, which is vital for the autonomous system to make informed decisions in real time.

In the realm of object detection, we direct our efforts toward refining the Faster R-CNN model. Recognized for its accuracy and detailed detection, the Faster R-CNN model is a powerful tool in the machine learning landscape. Our research utilizes the KITTI dataset, acknowledged for its rich variety of annotated driving scenes that closely mimic the complexities encountered on real roads. By working with such a realistic dataset, we aim to push the boundaries of our model's robustness and precision in object detection tasks.

The Faster R-CNN model was chosen for its proven effectiveness in identifying objects with a high degree of accuracy. This study aims to enhance the model's existing strengths by meticulously adjusting and improving its architecture. We seek to optimize its settings, with the end goal of achieving unparalleled performance that can withstand the varied challenges presented by different driving scenarios.

In our comprehensive study, we also examine other models, specifically YOLOv3 and YOLOv5, to gauge the breadth of current object detection capabilities. By analyzing these models alongside Faster R-CNN, we gain valuable insight into the comparative strengths and weaknesses of each approach. This evaluation is not merely academic; it serves as a practical guide for selecting the most appropriate model for specific autonomous driving tasks.

Our results indicate that while Faster R-CNN offers a high level of accuracy, its computational demands make it less than ideal for scenarios requiring immediate response. On the other hand, YOLO models, known for their speed in processing, offer a more suitable solution for real-time object detection despite their slightly lower accuracy compared to Faster R-CNN.

This paper aims to extend the discourse on object detection within the sphere of autonomous driving. By presenting our findings, we hope to aid in the meticulous selection and optimization of object detection models. Ensuring that these systems are not only effective but also efficient is paramount, as they must perform reliably under the diverse and dynamic conditions that characterize everyday driving scenarios.

In conclusion, the pursuit of this research is to enhance the understanding of object detection algorithms and their application in autonomous vehicles. With this knowledge, we aim to contribute to the development of more advanced, safe, and reliable autonomous driving systems that are well-equipped to handle the complexities of the road.

II. MOTIVATION

The YOLO algorithm occupies a significant position in the realm of object detection. In the course of our project "Multimodal Sensor Integration for the Karcher Autonomous Cleaning Robot," we aimed to utilize the KITTI dataset for simulation and testing of our product. However, the application revealed that the algorithm did not yield satisfactory predictive results, exhibiting considerable deviations from expected accuracies. Consequently, we are compelled to identify and adopt novel methodologies that can more aptly cater to our targeted scenarios, specifically the road environments encountered in autonomous driving applications. This forms the motivation

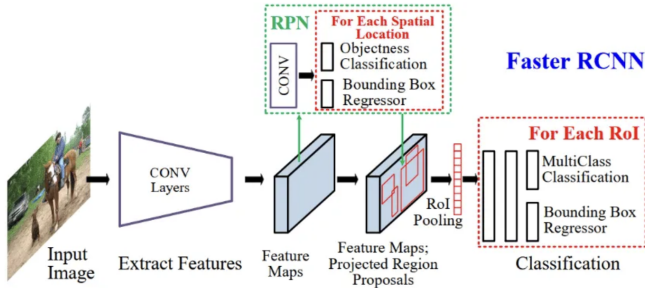


Fig. 1. Faster R-CNN overview

for our pursuit of a more robust detection system, tailored to the intricate demands of autonomous navigation and safety.

III. RELATED WORK

Following the pioneering work of Girshick et al. on the Faster R-CNN model for object detection, a transformative shift was observed in the object detection domain[1]. Key datasets for object detection like PASCAL VOC , ImageNet, and the more recent KITTI provided a platform for evaluating the robustness and efficiency of these models in diverse environments.[2]

With the availability of such rich datasets, the object detection realm saw a surge in innovations and advancements, encompassing both real-world images and more specialized scenarios. YOLO, introduced by Redmon et al., marked another milestone, emphasizing speed without severely compromising accuracy. Its subsequent versions, including YOLOv3 and YOLOv5 , further established its prominence in the real-time detection sector[3].

While models like Faster R-CNN focused extensively on accuracy with relatively more computational overhead , YOLO variants pushed the boundaries of what's achievable in real-time detection, albeit sometimes at the cost of precision.[4]

Recently, autonomous driving technologies have embraced these models with varying enthusiasm. Major players in the autonomous driving space have released datasets that incorporate both image and sensor data, such as LiDAR . While many of these datasets furnish annotations for objects through bounding boxes, only a handful extend to a detailed semantic level necessary for more granular tasks like the one tackled in our research .

A. Faster-RCNN

Faster R-CNN, or "Faster Region-based Convolutional Neural Network," stands as a cornerstone in the realm of object detection. Unlike mere image classification that classifies an entire image into a category, object detection aims to pinpoint the location of objects within an image while also classifying them. In this domain, Faster R-CNN has showcased its prowess by achieving state-of-the-art results.

The architecture operates by generating high-quality region proposals, which are then processed by a Convolutional Neural

TABLE I
RESULTS OF MAP FOR KITTI USING MODIFIED YOLOv3.

Benchmark	Easy	Moderate	Hard
Car	56.00%	36.23%	29.55%
Pedestrian	29.98%	22.84%	22.21%
Cyclist	9.09%	9.09%	9.09%

Network (CNN) for feature extraction. These extracted features are subsequently subjected to classification and bounding box regression. The system comprises two main modules. The first is the Region Proposal Network (RPN), which is entrusted with predicting object boundaries and objectness scores. The second is the Fast R-CNN detector, which employs the proposed regions for detection and classification tasks. A unique aspect of Faster R-CNN is the sharing of convolutional features between the RPN and the detection network, optimizing efficiency.[5]

Historically, the development of R-CNN models started with the original R-CNN. This model utilized selective search for region proposals but was computationally demanding because it processed each region separately. Then came Fast R-CNN, which introduced the concept of RoI (Region of Interest) pooling, allowing the sharing of convolutional features across all proposals. Faster R-CNN elevated this idea further by seamlessly integrating the RPN, making the region proposal step almost cost-neutral in computational terms.

However, every architecture has its trade-offs. While Faster R-CNN boasts of precision, efficient feature sharing, and versatility in handling diverse object sizes, it still doesn't match the speed of some other detection architectures like YOLO. YOLO, or "You Only Look Once," diverges from Faster R-CNN by merging the detection process into a single step, enhancing its speed but sometimes compromising precision.[6]

B. yolo

The object detection landscape witnessed a paradigm shift with the introduction of YOLO, an acronym for "You Only Look Once." Spearheaded by Joseph Redmon and his team, YOLO's unique proposition was its ability to detect and classify objects in images in a single forward pass of the neural network. This departure from the traditional two-step processes, such as those in R-CNN-based methods, allowed YOLO to achieve real-time detection speeds without substantially compromising accuracy.

YOLOv3, an advanced iteration of the YOLO series, brought about a series of enhancements. One of its notable features was the use of three different sizes of anchor boxes for each detection scale. This nuanced approach empowered YOLOv3 to better detect objects of varied sizes[7]. Additionally, it employed three different scales of detection, leveraging features from various layers of the network. This multi-scale detection strategy enhanced its precision across a range of object sizes. Another leap in YOLOv3 was its use of three prediction branches, each responsible for predicting bounding boxes, objectness scores, and class scores.

TABLE II
RESULTS OF MAP FOR KITTI USING MODIFIED YOLOv5.

Benchmark	Easy	Moderate	Hard
Car	88.17%	78.70%	69.45%
Pedestrian	60.44%	43.69%	43.06%
Cyclist	55.00%	39.29%	32.58%

TABLE III
RESULTS OF MAP FOR KITTI USING RETRAINED FASTER R-CNN.

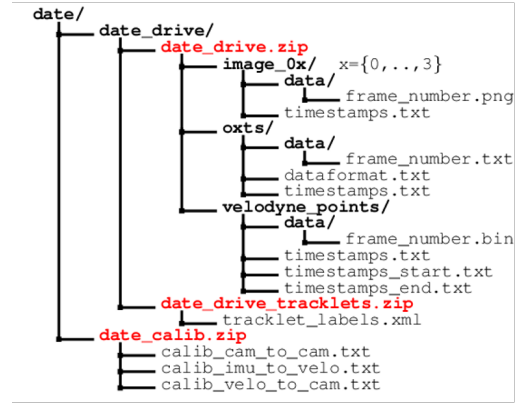
Benchmark	Easy	Moderate	Hard
Car	84.81%	86.18%	78.03%
Pedestrian	76.52%	59.98%	51.84%
Cyclist	74.72%	56.83%	49.60%

Progressing further, YOLOv5 emerged not as a direct successor from the original YOLO developers but as a community-driven evolution. Though its name might suggest otherwise, YOLOv5 does not have a v4 counterpart from the same lineage. YOLOv5, in comparison to YOLOv3, boasts improvements in terms of both speed and accuracy. With architectural tweaks, enhanced training techniques, and optimizations, it provides better performance metrics on various benchmarks[8].

IV. DATASET

The KITTI dataset, which stands for the Karlsruhe Institute of Technology and Toyota Technological Institute, has emerged as a cornerstone [9] in the computer vision community for the benchmarking of advanced algorithms in the scope of autonomous driving. This comprehensive dataset has been meticulously crafted to stimulate innovation in the development of sophisticated algorithms capable of interpreting complex real-world scenes, particularly those encountered in various driving conditions.

KITTI is not just a collection of high-resolution imagery; it encompasses a rich tapestry of data types, organized within a hierarchical structure that is conducive to a multitude of computer vision tasks, particularly object detection. The dataset is characterized by a range of high-resolution images, acquired in diverse driving environments, from densely populated urban areas to less-structured country roads. It systematically categorizes multiple object types, prominently including cars, pedestrians, and cyclists, each meticulously annotated with precise 2D and 3D bounding boxes. These annotations offer a multi-faceted view, revealing not only the location of objects within the image plane but also imparting extensive details about their spatial orientation and dimensions within a tangible real-world context. The KITTI dataset is uniquely structured, consisting of an organized collection within zip archives, segregated by date and specific driving sequences. The data repository encompasses a range of subdirectories, correlating to camera feeds, GPS and motion sensor information, as well as LiDAR point clouds – all critical for tasks such as depth perception and three-dimensional object detection. The inclusion of diverse sensor modalities affirms the dataset's comprehensive



Directory structure diagram of the data set

Fig. 2. The unique structure of the KITTI dataset

approach to supporting algorithmic development within the autonomous driving domain.[10][11][12][13][14]

Within these archives, the dataset is meticulously organized to include synchronized video frames, calibration parameters for sensor alignment, tracklets, and associated annotations. This detailed arrangement, including timestamps, calibration data for sensor fusion, and precise labeling, is indispensable for developing algorithms that rely on accurate spatial information obtained from two-dimensional imagery to understand and interpret three-dimensional space.

KITTI's incorporation of real-world driving scenarios, inclusive of varying lighting conditions, frequent occlusions, significant changes in viewpoint, and an array of scales, contributes to its reputation as a stringent and robust benchmark for object detection algorithms. The dataset's architecture and extensive labeling are deliberately designed to challenge and thereby improve the robustness of computer vision algorithms, ensuring their efficacy in real-world autonomous driving applications.

Since its inception, the KITTI dataset has played a pivotal role in the evolution of object detection and autonomous vehicle technologies. It has become a benchmark that not only tests but also drives forward the development of algorithms in this domain. Its real-world, varied, and challenging scenarios ensure that any algorithm validated against it can meet the diverse and rigorous demands of actual driving situations, making it an invaluable resource for researchers and practitioners alike.

V. METHODOLOGY

In the vast and continually evolving landscape of object detection, the choice of evaluation metrics and model architectures plays a pivotal role in determining the effectiveness of an approach. This section delves into the methodologies adopted in our study, highlighting the nuances of our evaluative and experimental procedures. We employ the Mean Average Precision (mAP) as our primary metric, providing a comprehensive assessment of detection performance across different object

categories and overlap thresholds. To discern the capabilities and potential limitations of contemporary detection architectures, we turn our attention to three models: the Faster R-CNN, known for its meticulous region proposal mechanism, and the YOLO (You Only Look Once) variants v3 and v5, celebrated for their speed and real-time applicability. [15] These models are rigorously tested on the KITTI object detection dataset, a benchmark that poses diverse challenges emblematic of real-world driving scenarios. Through this methodology, we aim to provide insights into the strengths and potential areas of improvement for each model in the context of automotive object detection.

A. Key Metrics for Evaluation

To comprehensively assess the efficacy of object detection models, it is crucial to consider metrics that encapsulate both precision and recall. The Mean Average Precision (mAP) is a comprehensive metric that reflects an algorithm's accuracy across various classes and is defined in terms of Precision (P) and Recall (R).

Precision (P) is the ratio of correctly predicted positive observations to the total predicted positives and is formally defined as:

$$P = \frac{TP}{TP + FP}$$

where TP represents True Positives and FP represents False Positives.

Recall (R) is the ratio of correctly predicted positive observations to all observations in the actual class and is defined as:

$$R = \frac{TP}{TP + FN}$$

where FN represents False Negatives.

For a fixed class, Average Precision (AP) is the area under the Precision-Recall (PR) curve, which plots Precision as a function of Recall:

$$AP = \int_0^1 P(R) dR$$

However, in practice, the PR curve is discrete, and AP is calculated as the weighted mean of precisions at each threshold, with the increase in recall from the previous threshold used as the weight:

$$AP = \sum_{n=1}^N (R_n - R_{n-1}) P_n$$

where P_n and R_n are the precision and recall at the n th threshold.

The mAP is then the mean of the Average Precision across all classes:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

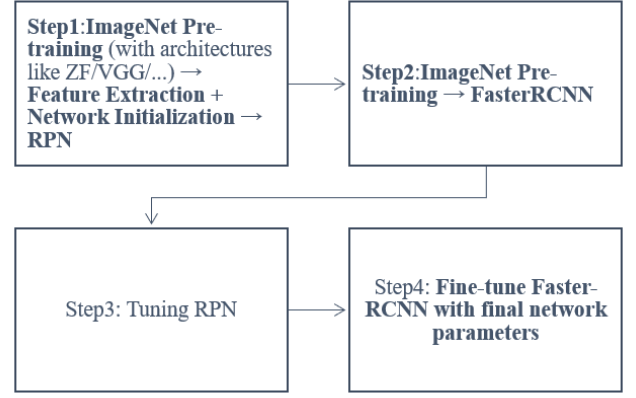


Fig. 3. Training and implementation details

where N is the number of classes and AP_i is the Average Precision for the i th class. It effectively captures the overall performance of the object detection algorithm across all classes, offering a single, coherent metric that considers both the existence and the precise localization of the objects.

In practical terms, mAP is calculated by first ordering all detections across all classes by their confidence scores. True Positives (TP) and False Positives (FP) are then accounted for at various recall levels, which are typically increased in small increments until the recall of 1 is reached. At each level, Precision is computed, and the AP for the class is the average of these values.

The mAP metric is particularly valuable in scenarios with imbalanced datasets where some classes are underrepresented. By taking the average across classes, mAP ensures that all classes, irrespective of their frequency, contribute equally to the overall effectiveness of the model.[16]

B. Training Details for Faster R-CNN

To harness the full potential of the Faster R-CNN for the KITTI dataset, we adopted a multi-stage training approach. Each phase was carefully designed to incrementally improve the model's performance on the complex real-world data presented in the KITTI benchmark.

1) *Stage 1: Region Proposal Network Initialization:* The Region Proposal Network (RPN) serves as the cornerstone[17] for identifying regions of interest. We initialized the RPN using weights from a model pre-trained on a comprehensive dataset, incorporating transfer learning to capitalize on previously learned features. This strategy expedited the convergence process and enhanced the RPN's ability to generate high-quality proposals tailored to the KITTI dataset's unique characteristics.

2) *Stage 2: Fast R-CNN Training:* Subsequent to the RPN initialization, we directed our efforts towards training the Fast R-CNN detector. Utilizing the proposals generated by the RPN, we trained the detector to classify objects within the proposed regions and refine their bounding boxes. This

YOLOv5



YOLOv3



Faster R-CNN



Fig. 4. Test examples in three situations

stage leveraged a freshly instantiated convolutional network, allowing for the customized adaptation of the model's weights to the specific nuances of the KITTI dataset, thereby enhancing detection accuracy.

3) *Stage 3: RPN Refinement*: Building on the outputs from the Fast R-CNN, we returned our focus to the RPN for further refinement. The objective was to iterate upon the network's ability to propose regions, drawing on the shared convolutional layers now optimized for KITTI-specific object detection. By keeping these layers fixed, we ensured stability in feature extraction, while honing the RPN's proposal mechanism for increased precision.

4) *Stage 4: Final Detector Fine-Tuning*: In the final phase, we fine-tuned the Fast R-CNN detector with a dual focus on leveraging the refined proposals from the RPN and maintaining the integrity of the shared convolutional features. This stage was critical for achieving the nuanced balance between proposal quality and detection capability, which is pivotal for high performance on the KITTI dataset.

The workflow encapsulated the setting up of the TensorFlow

Object Detection API, the conversion of the KITTI dataset into TensorFlow's tfrecord format, and the meticulous process of training and testing. With each iteration, we closely monitored loss metrics and visualized bounding box predictions to iteratively refine our model to below a 0.1 loss threshold, indicative of a well-trained Faster R-CNN model for the KITTI object detection task.

C. Training Details for YOLOv3 and YOLOv5

1) *Adaptation for KITTI Dataset*: Our training strategy for YOLO variants was finely tuned to the KITTI dataset's unique structure. YOLOv3 and YOLOv5 were adapted to handle the wide aspect ratio of the images.[18][19] We avoided the distortion of bounding boxes by modifying the input resolution parameters to match the dataset's native aspect ratio, thus ensuring the integrity of the object detection process.

2) *Configuration Optimization*: Both YOLO models underwent a series of configuration adjustments to optimize the network's architecture for the KITTI dataset. These optimizations included specific alterations to the input size to maintain the

original aspect ratio of images and adjustments to the filter sizes in the detection layers to accurately reflect the number of classes. Additionally, we disabled random resizing to provide a stable and consistent input for the models.

3) *Hyperparameter Tuning*: Further tuning of hyperparameters was conducted to enhance the models' performance.[20] Learning rates, object scales, and thresholds were calibrated for optimal detection precision. This fine-tuning process was critical in ensuring that the models were not only tailored to the dataset's image dimensions but also to its object characteristics.

These adjustments were instrumental in refining the performance of YOLOv3 and YOLOv5 on the KITTI dataset, leading to an improved object detection model that is both accurate and efficient.

VI. RESULT AND ANALYSIS

The subsequent illustration presents exemplary testing outcomes derived from the three models under consideration: Faster R-CNN, YOLOv3, and YOLOv5. For this empirical assessment, we handpicked three quintessential road scenarios from the KITTI dataset, characterized by a plethora of vehicles, pedestrians, and a diverse array of object classes. As depicted in the figure, Faster R-CNN demonstrates superior performance compared to both YOLO variants. In the highlighted instance, YOLO models manifest a noticeable limitation in detecting individuals situated on the left-hand side and manage to identify merely a single pedestrian on the right. In stark contrast, Faster R-CNN adeptly recognizes multiple pedestrians on the right, underscoring its enhanced detection capability in complex environments.

A. Qualitative results

Our comprehensive quantitative evaluation, stratified across easy, moderate, and hard datasets, revealed insightful patterns about the performance of the detection algorithms. In the easier dataset, where conditions were most favorable, Faster R-CNN demonstrated exceptional precision with cars, achieving a 88.17% detection rate compared to 56.00% for YOLOv3 and 84.81% for YOLOv5. However, for pedestrians, YOLOv5 outperformed its counterparts with a 60% detection rate, whereas Faster R-CNN achieved only 29.98%, highlighting the variability in performance across different object categories.

As the complexity of the dataset increased to moderate, Faster R-CNN maintained its lead in car detection at 86.18%, a marginal decrease, demonstrating its resilience to changing conditions. Contrastingly, YOLOv3 and YOLOv5 experienced more significant drops to 36.23% and 78.70% respectively. This trend persisted in the detection of cyclists, where Faster R-CNN excelled with 56.83%, while YOLOv5 held at 59.98%, and YOLOv3 significantly lagged behind with a mere 9.09% detection rate.

The hard dataset, characterizing the most challenging scenarios, continued to underscore the superior performance of Faster R-CNN, especially in the detection of cars with a

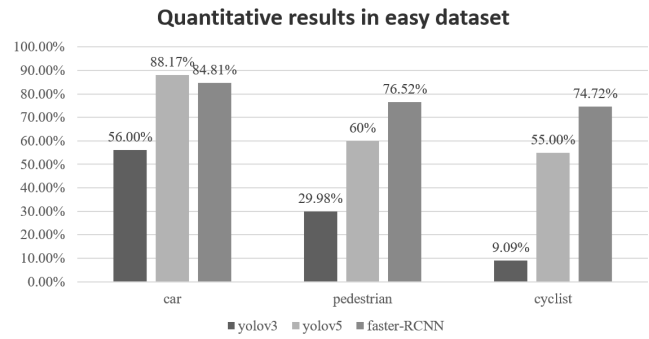


Fig. 5. Test examples in three situations

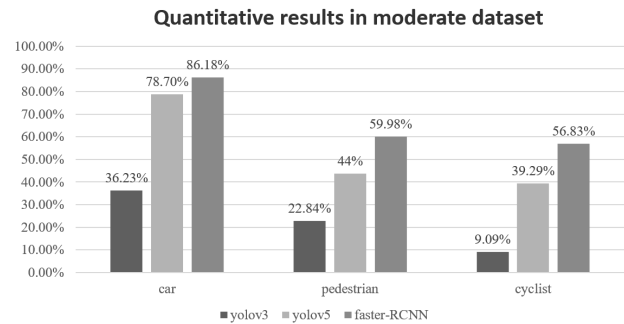


Fig. 6. Test examples in three situations

78.03% success rate. YOLOv5's performance was also noteworthy, particularly when detecting pedestrians with a 51.84% success rate, surpassing Faster R-CNN's 43%. However, the performance of YOLOv3 plummeted across all categories, bottoming at 29.55% for cars and an alarming 9.09% for both pedestrians and cyclists.

The consistency of Faster R-CNN's performance across all levels of difficulty underscores its robust feature extraction and region proposal capabilities, particularly when handling complex scenes with varied objects. Conversely, while YOLOv5 demonstrated notable adaptability, outperforming YOLOv3 and even Faster R-CNN in certain instances, its results were not as consistent across categories and datasets. These observations point to an inherent trade-off between the high-speed processing of YOLO architectures and the meticulous region-based processing of Faster R-CNN, which, while slower, provides enhanced precision in object detection tasks.

In conclusion, our data-driven analysis confirms Faster R-CNN as the more robust and accurate model for detecting objects in diverse and challenging environments, a critical requirement for autonomous driving systems. Its consistent outperformance across varying conditions and object categories establishes it as the preferred model for applications where reliability and precision are paramount.

Quantitative results in hard dataset

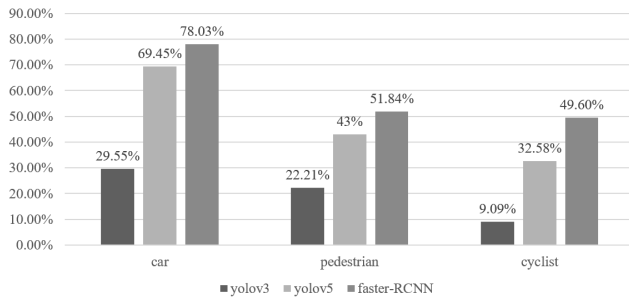


Fig. 7. Test examples in three situations

TABLE IV
INFERENCE TIME (PER FRAME) FOR EACH MODEL

Model	YOLOv3	YOLOv5	Faster R-CNN
Time	15 ms	35 ms	2763 ms

B. Execution time analysis

In addition to assessing detection accuracy, we conducted a meticulous evaluation of the execution time associated with each of the three models. Both YOLOv3 and YOLOv5 are posited as real-time detection frameworks. Our experiments corroborated this claim, as, for the KITTI dataset, both models were capable of completing object detection in under 40 ms per image. Notably, YOLOv5 demonstrated a slight temporal lag compared to its predecessor, YOLOv3.

Contrastingly, the Faster R-CNN, despite its moniker suggesting rapidity, exhibited a considerably extended execution time in comparison to the YOLO series. This temporal elongation renders the Faster R-CNN less suitable for applications demanding real-time response, such as autonomous driving. Even though Faster R-CNN outstripped the YOLO models in terms of detection performance, its latency compromises its feasibility for time-critical tasks. This dichotomy underscores the importance of carefully selecting appropriate models based on the specific demands and constraints of a given application.

VII. CONCLUSION

Throughout our investigation, we systematically applied three eminent object detection architectures - YOLOv3, YOLOv5, and Faster R-CNN - to the KITTI 2D object detection dataset. This endeavor encompassed meticulous data preprocessing, adaptive model retraining, and rigorous performance evaluations post-inference. Among the trio, Faster R-CNN demonstrated superior detection accuracy on the KITTI dataset, a testament to its sophisticated detection mechanism. However, its computational intensity and consequent lag in execution speed render it less suitable for real-time operations, particularly in dynamic environments like autonomous driving. While its precision is commendable, the quest for an optimal balance between speed and accuracy remains paramount in real-world applications.

REFERENCES

- [1] Chen, Yuhua, et al. "Domain adaptive faster r-cnn for object detection in the wild." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.
- [2] Akhtar, Malik Javed, et al. "A Robust Framework for Object Detection in a Traffic Surveillance System." Electronics 11.21 (2022): 3425.
- [3] Haris, Malik, and Adam Glowacz. "Road object detection: A comparative study of deep learning-based algorithms." Electronics 10.16 (2021): 1932.
- [4] Huang, Rachel, Jonathan Pedoeem, and Cuixian Chen. "YOLO-LITE: a real-time object detection algorithm optimized for non-GPU computers." 2018 IEEE international conference on big data (big data). IEEE, 2018.
- [5] Ren, Shaoqing, et al. "Faster r-cnn: Towards real-time object detection with region proposal networks." Advances in neural information processing systems 28 (2015).
- [6] Wang, Jiayuan, Q. M. Wu, and Ning Zhang. "You Only Look at Once for Real-time and Generic Multi-Task." arXiv preprint arXiv:2310.01641 (2023).
- [7] Li, Hao, et al. "Enhanced YOLO v3 tiny network for real-time ship detection from visual image." Ieee Access 9 (2021): 16692-16706.
- [8] Yu, Feiyang, et al. "Improved YOLO-v5 model for boosting face mask recognition accuracy on heterogeneous IoT computing platforms." Internet of Things 23 (2023): 100881.
- [9] Gupta, Abhishek, et al. "Deep learning for object detection and scene perception in self-driving cars: Survey, challenges, and open issues." Array 10 (2021): 100057.
- [10] Huang, Xinyu, et al. "The apolloscape open dataset for autonomous driving and its application." IEEE transactions on pattern analysis and machine intelligence 42.10 (2019): 2702-2719.
- [11] Al-refai, Ghaith, and Mohammed Al-refai. "Road Object Detection using Yolov3 and Kitti Dataset." International Journal of Advanced Computer Science and Applications 11.8 (2020).
- [12] Geiger, Andreas, et al. "Vision meets robotics: The kitti dataset." The International Journal of Robotics Research 32.11 (2013): 1231-1237.
- [13] Zhang, Yigong, et al. "Dijkstra model for stereo-vision based road detection: a non-parametric method." 2018 IEEE International conference on robotics and automation (ICRA). IEEE, 2018.
- [14] Arnold, Eduardo, et al. "A survey on 3d object detection methods for autonomous driving applications." IEEE Transactions on Intelligent Transportation Systems 20.10 (2019): 3782-3795.
- [15] Park, Sung-Sik, Van-Than Tran, and Dong-Eun Lee. "Application of various yolo models for computer vision-based real-time pothole detection." Applied Sciences 11.23 (2021): 11229.
- [16] Henderson, Paul, and Vittorio Ferrari. "End-to-end training of object class detectors for mean average precision." Computer Vision-ACCV 2016: 13th Asian Conference on

Computer Vision, Taipei, Taiwan, November 20-24, 2016, Revised Selected Papers, Part V 13. Springer International Publishing, 2017.

[17] Zhu, Li, et al. "Iou-uniform r-cnn: Breaking through the limitations of rpn." *Pattern Recognition* 112 (2021): 107816.

[18]Zhang, Heng, et al. "Research on Object Detection and Recognition Method for UAV Aerial Images Based on Improved YOLOv5." *Drones* 7.6 (2023): 402.

[19] Sozzi, Marco, et al. "Automatic bunch detection in white grape varieties using YOLOv3, YOLOv4, and YOLOv5 deep learning algorithms." *Agronomy* 12.2 (2022): 319.

[20] Schratz, Patrick, et al. "Hyperparameter tuning and performance assessment of statistical and machine-learning algorithms using spatial data." *Ecological Modelling* 406 (2019): 109-120.